O'REILLY®

Second Edition

Head First Design Patterns

Building Extensible & Maintainable Object-Oriented Software

Eric Freeman & Elisabeth Robson

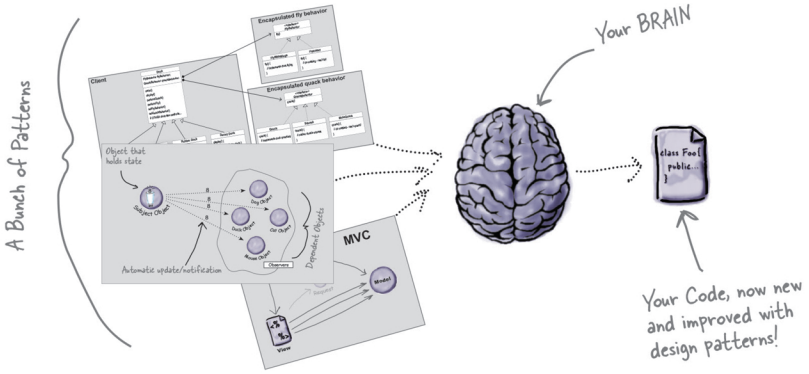
with Kathy Sierra & Bert Bates

A Brain-Friendly Guide

Head First
Design Patterns

What will you learn from this book?

You know you don't want to reinvent the wheel, so you look to Design Patterns—the lessons learned by those who've faced the same software design problems. With Design Patterns, you get to take advantage of the best practices and experience of others so you can spend your time on something more challenging. Something more fun. This book shows you the patterns that matter, when to use them and why, how to apply them to your own designs, and the object-oriented design principles on which they're based. Join hundreds of thousands of developers who've improved their object-oriented design skills through *Head First Design Patterns*.



What's so special about this book?

If you've read a Head First book, you know what to expect—a visually rich format designed for the way your brain works. With *Head First Design Patterns, 2E* you'll learn design principles and patterns in a way that won't put you to sleep, so you can get out there to solve software design problems and speak the language of patterns with others on your team.

"I received the book yesterday and started to read it...and I couldn't stop. This is très 'cool.' It is fun, but they cover a lot of ground and they are right to the point. I'm really impressed."

—Erich Gamma
IBM Distinguished Engineer, and coauthor of *Design Patterns*

"I feel like a thousand pounds of books have just been lifted off of my head."

—Ward Cunningham
inventor of the Wiki and founder of the Hillside Group

"*Head First Design Patterns* manages to mix fun, belly laughs, insight, technical depth, and great practical advice in one entertaining and thought-provoking read."

—Richard Helm
coauthor of *Design Patterns*

COMPUTER PROGRAMMING

US $69.99 **CAN $92.99**

ISBN: 978-1-492-07800-5

O'REILLY®

Praise for *Head First Design Patterns*

"I received the book yesterday and started to read it on the way home…and I couldn't stop. I took it to the gym and I expect people saw me smiling a lot while I was exercising and reading. This is très 'cool'. It is fun, but they cover a lot of ground and they are right to the point. I'm really impressed."

— **Erich Gamma, IBM Distinguished Engineer and coauthor of *Design Patterns* with the rest of the Gang of Four–Richard Helm, Ralph Johnson, and John Vlissides**

"*Head First Design Patterns* manages to mix fun, belly-laughs, insight, technical depth, and great practical advice in one entertaining and thought-provoking read. Whether you are new to Design Patterns or have been using them for years, you are sure to get something from visiting Objectville."

— **Richard Helm, coauthor of *Design Patterns* with the rest of the Gang of Four–Erich Gamma, Ralph Johnson, and John Vlissides**

"I feel like a thousand pounds of books have just been lifted off of my head."

— **Ward Cunningham, inventor of the Wiki and founder of the Hillside Group**

"This book is close to perfect, because of the way it combines expertise and readability. It speaks with authority and it reads beautifully. It's one of the very few software books I've ever read that strikes me as indispensable. (I'd put maybe 10 books in this category, at the outside.)"

— **David Gelernter, Professor of Computer Science, Yale University, and author of *Mirror Worlds* and *Machine Beauty***

"A Nose Dive into the realm of patterns, a land where complex things become simple, but where simple things can also become complex. I can think of no better tour guides than Eric and Elisabeth."

— **Miko Matsumura, Industry Analyst, The Middleware Company Former Chief Java Evangelist, Sun Microsystems**

"I laughed, I cried, it moved me."

— **Daniel Steinberg, Editor-in-Chief, java.net**

"My first reaction was to roll on the floor laughing. After I picked myself up, I realized that not only is the book technically accurate, it is the easiest-to-understand introduction to Design Patterns that I have seen."

— **Dr. Timothy A. Budd, Associate Professor of Computer Science at Oregon State University and author of more than a dozen books, including *C++ for Java Programmers***

"Jerry Rice runs patterns better than any receiver in the NFL, but Eric and Elisabeth have outrun him. Seriously…this is one of the funniest and smartest books on software design I've ever read."

— **Aaron LaBerge, SVP Technology & Product Development, ESPN**

More Praise for *Head First Design Patterns*

"Great code design is, first and foremost, great information design. A code designer is teaching a computer how to do something, and it is no surprise that a great teacher of computers should turn out to be a great teacher of programmers. This book's admirable clarity, humor, and substantial doses of clever make it the sort of book that helps even non-programmers think well about problem-solving."

— **Cory Doctorow, coeditor of Boing Boing and author of *Down and Out in the Magic Kingdom* and *Someone Comes to Town, Someone Leaves Town***

"There's an old saying in the computer and videogame business—well, it can't be that old because the discipline is not all that old—and it goes something like this: Design is Life. What's particularly curious about this phrase is that even today almost no one who works at the craft of creating electronic games can agree on what it means to 'design' a game. Is the designer a software engineer? An art director? A storyteller? An architect or a builder? A pitch person or a visionary? Can an individual indeed be in part all of these? And most importantly, who the %$!#&* cares?

It has been said that the 'designed by' credit in interactive entertainment is akin to the 'directed by' credit in filmmaking, which in fact allows it to share DNA with perhaps the single most controversial, overstated, and too often entirely lacking in humility credit grab ever propagated on commercial art. Good company, eh? Yet if Design is Life, then perhaps it is time we spent some quality cycles thinking about what it is.

Eric Freeman and Elisabeth Robson have intrepidly volunteered to look behind the code curtain for us in *Head First Design Patterns*. I'm not sure either of them cares all that much about the PlayStation or Xbox, nor should they. Yet they do address the notion of design at a significantly honest level such that anyone looking for ego reinforcement of his or her own brilliant auteurship is best advised not to go digging here where truth is stunningly revealed. Sophists and circus barkers need not apply. Next-generation literati, please come equipped with a pencil."

— **Ken Goldstein, Executive Vice President & Managing Director, Disney Online**

"This is a difficult blurb for me to write since Eric and Elisabeth were my students a long time ago, so I don't want to be seen to be too drooling, but this is the best book on Design Patterns available for students. As proof: I have used it ever since it was published, in both in my grad and undergrad courses, both for software engineering and advanced programming. As soon as it came out I abandoned the Gang of Four as well as all competitors!"

— **Gregory Rawlins, Indiana University**

"This book combines good humor, great examples, and in-depth knowledge of Design Patterns in such a way that makes learning fun. Being in the entertainment technology industry, I am intrigued by the Hollywood Principle and the home theater Facade Pattern, to name a few. The understanding of Design Patterns not only helps us create reusable and maintainable quality software, but also helps sharpen our problem-solving skills across all problem domains. This book is a must-read for all computer professionals and students."

— **Newton Lee, Founder and Editor-in-Chief, Association for Computing Machinery's (ACM) Computers in Entertainment (acmcie.org)**

Praise for other books by Eric Freeman and Elisabeth Robson

"I literally love this book. In fact, I kissed this book in front of my wife."

— **Satish Kumar**

"*Head First HTML and CSS* is a thoroughly modern introduction to forward-looking practices in web page markup and presentation. It correctly anticipates readers' puzzlements and handles them just in time. The highly graphic and incremental approach precisely mimics the best way to learn this stuff: make a small change and see it in the browser to understand what each new item means."

— **Danny Goodman, author of *Dynamic HTML: The Definitive Guide***

"The Web would be a much better place if every HTML author started off by reading this book."

— **L. David Baron, Technical Lead, Layout & CSS, Mozilla Corporation**
http://dbaron.org

"My wife stole the book. She's never done any web design, so she needed a book like *Head First HTML and CSS* to take her from beginning to end. She now has a list of websites she wants to build—for our son's class, our family…If I'm lucky, I'll get the book back when she's done."

— **David Kaminsky, Master Inventor, IBM**

"This book takes you behind the scenes of JavaScript and leaves you with a deep understanding of how this remarkable programming language works. I wish I'd had *Head First JavaScript Programming* when I was starting out!"

— **Chris Fuselier, engineering consultant**

"The *Head First* series utilizes elements of modern learning theory, including constructivism, to bring readers up to speed quickly. The authors have proven with this book that expert-level content can be taught quickly and efficiently. Make no mistake here, this is a serious JavaScript book, and yet, fun reading!"

— **Frank Moore, web designer and developer**

"Looking for a book that will keep you interested (and laughing) but teach you some serious programming skills? *Head First JavaScript Programming* is it!"

— **Tim Williams, software entrepreneur**

Other O'Reilly books by Eric Freeman and Elisabeth Robson

Head First Learn to Code

Head First JavaScript Programming

Head First HTML and CSS

Head First HTML5 Programming

Other related books from O'Reilly

Head First Java

Learning Java

Java in a Nutshell

Java Enterprise in a Nutshell

Java Examples in a Nutshell

Java Cookbook

J2EE Design Patterns

Head First
Design Patterns

Eric Freeman
Elisabeth Robson

O'REILLY®

Beijing • Boston • Farnham • Sebastopol • Tokyo

Head First Design Patterns, 2nd Edition

by Eric Freeman, Elisabeth Robson, Kathy Sierra, and Bert Bates

Printed in Canada.

Published by O'Reilly Media, Inc., 1005 Gravenstein Highway North, Sebastopol, CA 95472.

O'Reilly Media books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (*oreilly.com*). For more information, contact our corporate/institutional sales department: (800) 998-9938 or *corporate@oreilly.com*.

Editors 1st Edition: Mike Hendrickson, Mike Loukides

Editors 2nd Edition: Michele Cronin, Melissa Duffield

Cover Designer: Ellie Volckhausen

Pattern Wranglers: Eric Freeman, Elisabeth Robson

Printing History:

October 2004: First edition

December 2020: Second edition

Release History:

2020-11-10 First release

In other words, if you use anything in *Head First Design Patterns* to, say, run a nuclear power plant, you're on your own. We do, however, encourage you to use the DJ View app.

No ducks were harmed in the making of this book.

The original GoF agreed to have their photos in this book. Yes, they really are that good-looking.

ISBN: 978-1-492-07800-5

[MBP]

To the Gang of Four, whose insight and expertise in capturing and communicating Design Patterns has changed the face of software design forever, and bettered the lives of developers throughout the world.

But *seriously*, when are we going to see a second edition? After all, it's been only ~~*ten*~~ twenty-five *years*.

Authors of Head First Design Patterns

Eric Freeman

Elisabeth Robson

Eric is described by Head First series co-creator Kathy Sierra as "one of those rare individuals fluent in the language, practice, and culture of multiple domains from hipster hacker, corporate VP, engineer, think tank."

By training, Eric is a computer scientist, having earned his PhD at Yale University. Professionally, Eric was formerly CTO of Disney Online & Disney.com at the Walt Disney Company.

Eric now co-directs the Head First series and devotes his time to creating print and video content at WickedlySmart, which is distributed across the leading educational channels.

Eric's Head First titles include *Head First Design Patterns*, *Head First HTML & CSS*, *Head First JavaScript Programming*, *Head First HTML5 Programming*, and *Head First Learn to Code*.

Eric lives in Austin, Texas.

Elisabeth is a software engineer, writer, and trainer. She has been passionate about technology since her days as a student at Yale University, where she earned a Masters of Science in Computer Science.

She's currently cofounder of WickedlySmart, where she creates books, articles, videos, and more. Previously, as Director of Special Projects at O'Reilly Media, Elisabeth produced in-person workshops and online courses on a variety of technical topics and developed her passion for creating learning experiences to help people understand technology.

When not in front of her computer, you'll find Elisabeth hiking, cycling, kayaking, and gardening in the great outdoors, often with her camera nearby.

Creators of the Head First Series

Kathy has been interested in learning theory since her days as a game designer for Virgin, MGM, and Amblin', and a teacher of New Media Authoring at UCLA. She was a master Java trainer for Sun Microsystems, and she founded JavaRanch.com (now CodeRanch.com), which won Jolt Cola Productivity awards in 2003 and 2004.

In 2015, she won the Electronic Frontier Foundation's Pioneer Award for her work creating skillful users and building sustainable communities.

Kathy's recent focus has been on cutting-edge, movement science and skill acquisition coaching, known as ecological dynamics or "Eco-D." Her work using Eco-D for training horses is ushering in a far, far more humane approach to horsemanship, causing delight for some (and sadly, consternation for others). Those fortunate (autonomous!) horses whose owners are using Kathy's approach are happier, healthier, and more athletic than their fellows who are traditionally trained.

You can follow Kathy on Instagram: @pantherflows.

Before **Bert** was an author, he was a developer, specializing in old-school AI (mostly expert systems), real-time OSes, and complex scheduling systems.

In 2003, Bert and Kathy wrote *Head First Java* and launched the Head First series. Since then, he's written more Java books, and consulted with Sun Microsystems and Oracle on many of their Java certifications. He's also trained hundreds of authors and editors to create books that teach well.

Bert's a Go player, and in 2016 he watched in horror and fascination as AlphaGo trounced Lee Sedol. Recently he's been using Eco-D (ecological dynamics) to improve his golf game and to train his parrotlet Bokeh.

Bert and Kathy have been privileged to know Beth and Eric for 16 years now, and the Head First series is extremely fortunate to count them as key contributors.

You can send Bert a message at CodeRanch.com.

Table of Contents (summary)

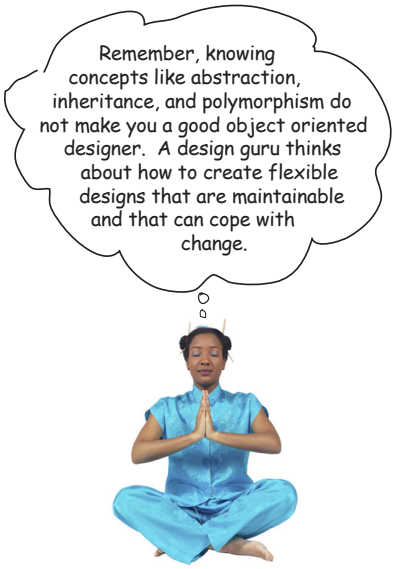
Table of Contents (the real thing)

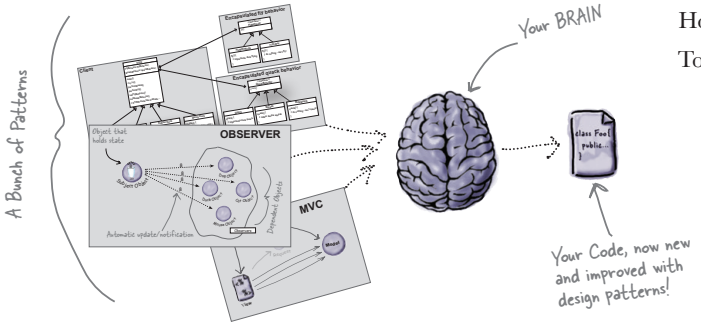
Intro

Your brain on Design Patterns. Here *you* are trying to *learn* something, while here your *brain* is doing you a favor by making sure the learning doesn't *stick*. Your brain's thinking, "Better leave room for more important things, like which wild animals to avoid and whether naked snowboarding is a bad idea." So how *do* you trick your brain into thinking that your life depends on knowing Design Patterns?

1 Welcome to Design Patterns

Someone has already solved your problems. In this chapter, you'll learn why (and how) you can exploit the wisdom and lessons learned by other developers who've been down the same design problem road and survived the trip. Before we're done, we'll look at the use and benefits of design patterns, look at some key object-oriented (OO) design principles, and walk through an example of how one pattern works. The best way to use patterns is to *load your brain* with them and then *recognize places* in your designs and existing applications where you can *apply them*. Instead of *code* reuse, with patterns you get *experience* reuse.

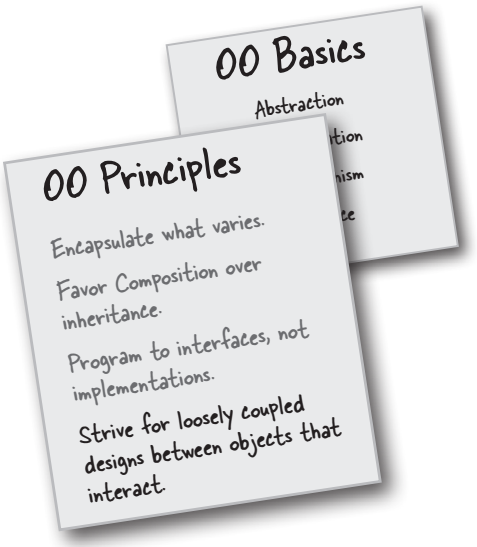


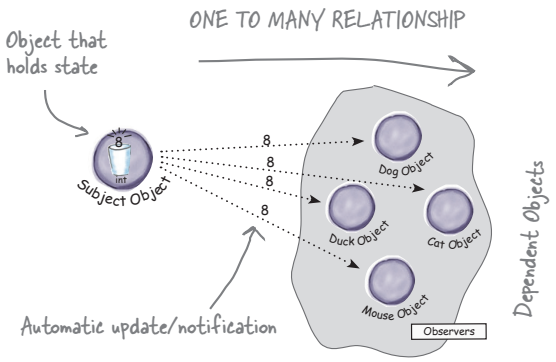


the Observer Pattern

2 Keeping your Objects in the Know

You don't want to miss out when something interesting happens, do you? We've got a pattern that keeps your objects *in the know* when something they *care about* happens. It's the Observer Pattern. It is one of the most commonly used design patterns, and it's incredibly useful. We're going to look at all kinds of interesting aspects of Observer, like its *one-to-many relationships* and *loose coupling*. And, with those concepts in mind, how can you help but be the life of the Patterns Party?

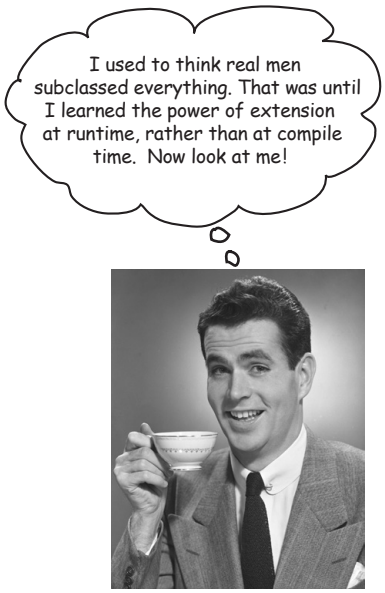




the Decorator Pattern

3 Decorating Objects

Just call this chapter "Design Eye for the Inheritance Guy." We'll re-examine the typical overuse of inheritance and you'll learn how to decorate your classes at runtime using a form of object composition. Why? Once you know the techniques of decorating, you'll be able to give your (or someone else's) objects new responsibilities *without making any code changes to the underlying classes.*

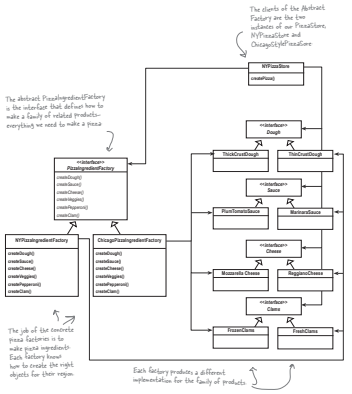


the Factory Pattern

4 Baking with OO Goodness

Get ready to bake some loosely coupled OO designs.

There is more to making objects than just using the **new** operator. You'll learn that instantiation is an activity that shouldn't always be done in public and can often lead to *coupling problems*. And we don't want *that*, do we? Find out how Factory Patterns can help save you from embarrassing dependencies.



the Singleton Pattern

5 One-of-a-Kind Objects

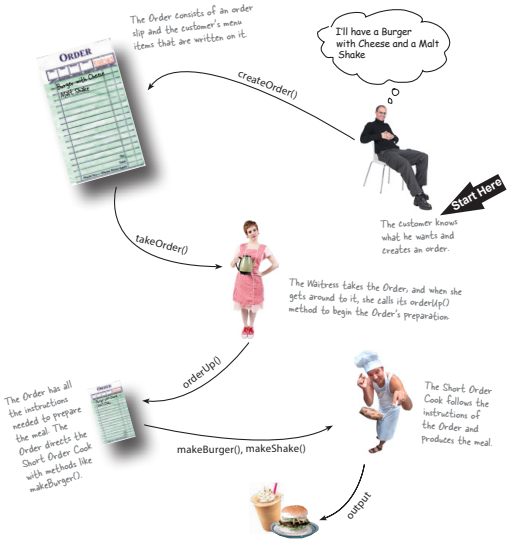
Our next stop is the Singleton Pattern, our ticket to creating one-of-a-kind objects for which there is only one instance, ever. You might be happy to know that of all patterns, the Singleton is the simplest in terms of its class diagram; in fact, the diagram holds just a single class! But don't get too comfortable; despite its simplicity from a class design perspective, it's going to require some deep object-oriented thinking in its implementation. So put on that thinking cap, and let's get going.

the Command Pattern

6 Encapsulating Invocation

In this chapter, we take encapsulation to a whole new level: we're going to encapsulate method invocation.

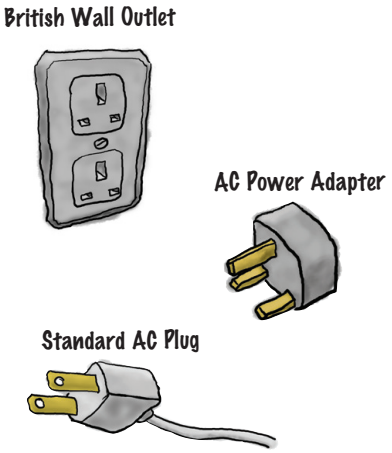
That's right—by encapsulating method invocation, we can crystallize pieces of computation so that the object invoking the computation doesn't need to worry about how to do things, it just uses our crystallized method to get it done. We can also do some wickedly smart things with these encapsulated method invocations, like save them away for logging or reuse them to implement undo functionality in our code.

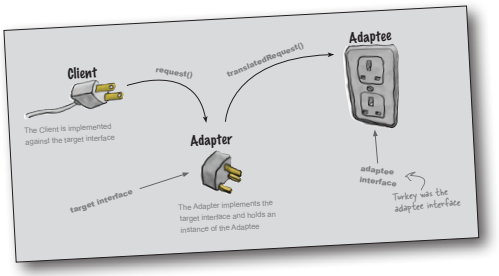


the Adapter and Facade Patterns

7 Being Adaptive

In this chapter we're going to attempt such impossible feats as putting a square peg in a round hole. Sound impossible? Not when we have Design Patterns. Remember the Decorator Pattern? We **wrapped objects** to give them new responsibilities. Now we're going to wrap some objects with a different purpose: to make their interfaces look like something they're not. Why would we do that? So we can adapt a design expecting one interface to a class that implements a different interface. That's not all; while we're at it, we're going to look at another pattern that wraps objects to simplify their interface.





the Template Method Pattern

8 Encapsulating Algorithms

We've encapsulated object creation, method invocation, complex interfaces, ducks, pizzas…what could be next?

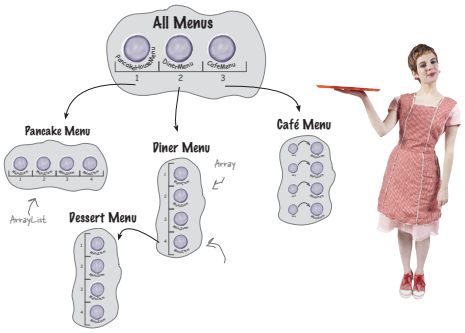
We're going to get down to encapsulating pieces of algorithms so that subclasses can hook themselves right into a computation anytime they want. We're even going to learn about a design principle inspired by Hollywood. Let's get started…

the Iterator and Composite Patterns

9 Well-Managed Collections

There are lots of ways to stuff objects into a collection.

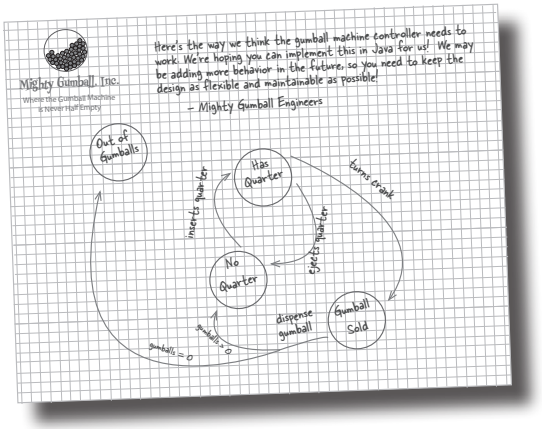
Put them into an Array, a Stack, a List, a hash map—take your pick. Each has its own advantages and tradeoffs. But at some point your clients are going to want to iterate over those objects, and when they do, are you going to show them your implementation? We certainly hope not! That just wouldn't be professional. Well, you don't have to risk your career; in this chapter you're going to see how you can allow your clients to iterate through your objects without ever getting a peek at how you store your objects. You're also going to learn how to create some super collections of objects that can leap over some impressive data structures in a single bound. And if that's not enough, you're also going to learn a thing or two about object responsibility.



the State Pattern

10 The State of Things

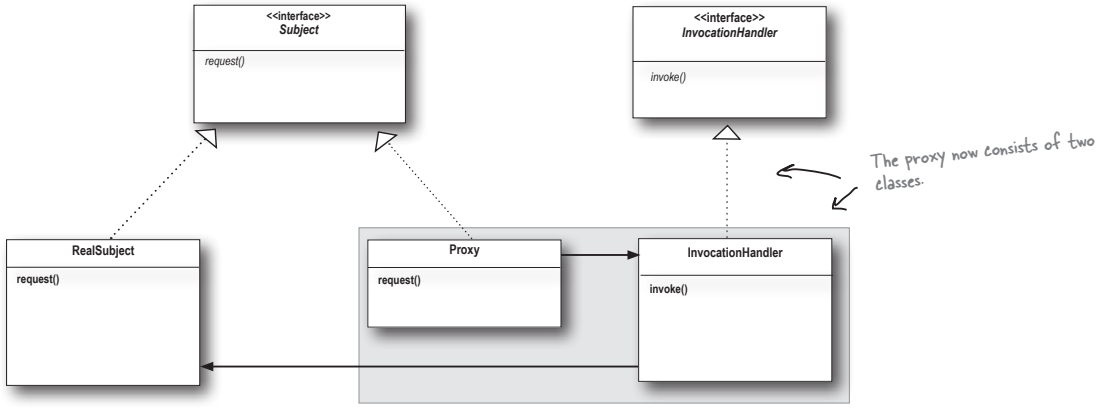
A little-known fact: the Strategy and State Patterns were twins separated at birth. You'd think they'd live similar lives, but the Strategy Pattern went on to create a wildly successful business around interchangeable algorithms, while State took the perhaps more noble path of helping objects to control their behavior by changing their internal state. As different as their paths became, however, underneath you'll find almost precisely the same design. How can that be? As you'll see, Strategy and State have very different intents. First, let's dig in and see what the State Pattern is all about, and then we'll return to explore their relationship at the end of the chapter.



the Proxy Pattern

11 Controlling Object Access

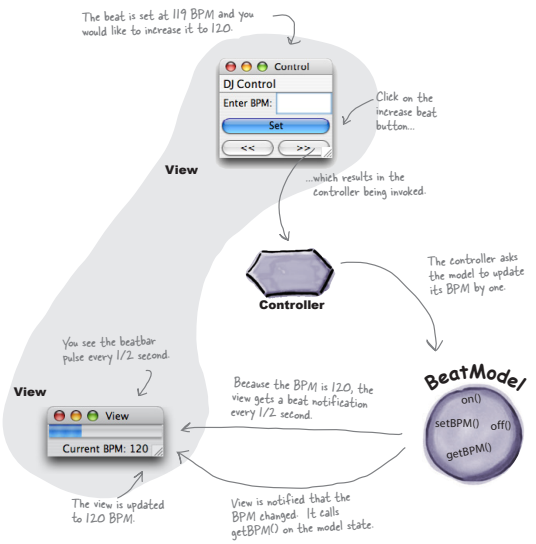
Ever play good cop, bad cop? You're the good cop and you provide all your services in a nice and friendly manner, but you don't want everyone asking you for services, so you have the bad cop control access to you. That's what proxies do: control and manage access. As you're going to see, there are lots of ways in which proxies stand in for the objects they proxy. Proxies have been known to haul entire method calls over the internet for their proxied objects; they've also been known to patiently stand in for some pretty lazy objects.

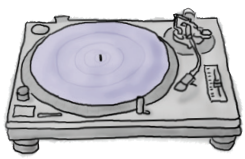


compound patterns

12 Patterns of Patterns

Who would have ever guessed that Patterns could work together? You've already witnessed the acrimonious Fireside Chats (and you haven't even seen the Pattern Death Match pages that the editor forced us to remove from the book), so who would have thought patterns can actually get along well together? Well, believe it or not, some of the most powerful OO designs use several patterns together. Get ready to take your pattern skills to the next level; it's time for compound patterns.

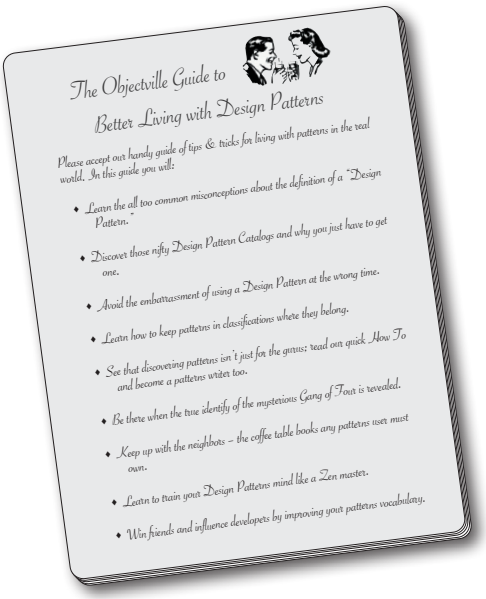


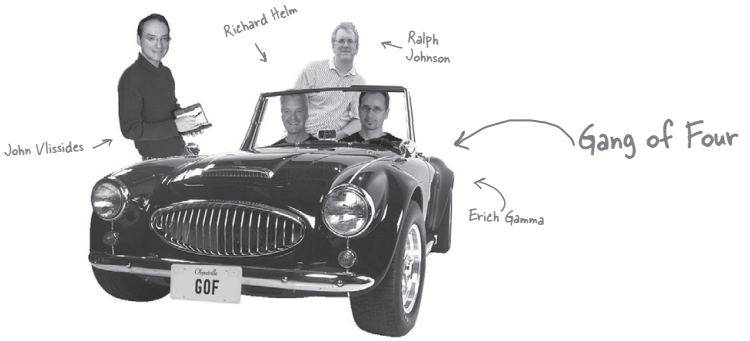


better living with patterns

13 Patterns in the Real World

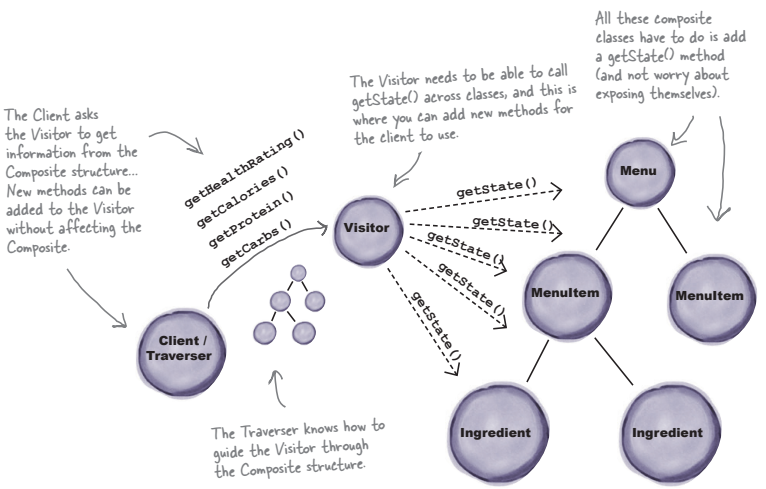
Ahhhh, now you're ready for a bright new world filled with Design Patterns. But, before you go opening all those new doors of opportunity, we need to cover a few details that you'll encounter out in the real world—that's right, things get a little more complex than they are here in Objectville. Come along, we've got a nice guide to help you through the transition…



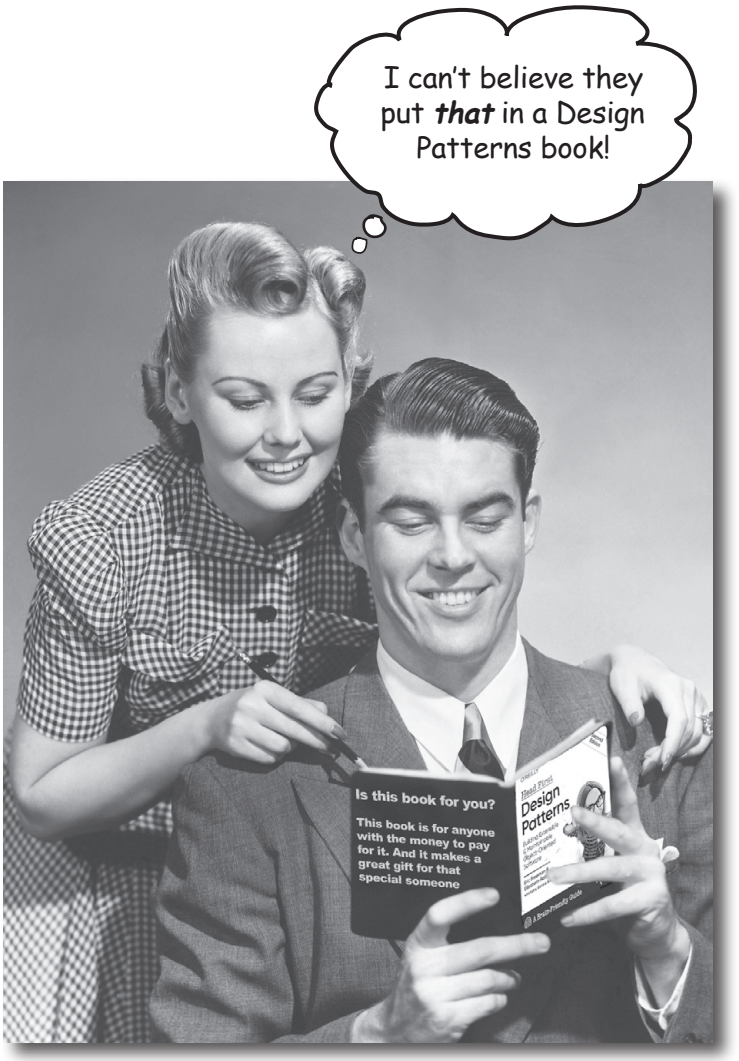


14 Appendix: Leftover Patterns

Not everyone can be the most popular. A lot has changed in the last 25+ years. Since *Design Patterns: Elements of Reusable Object-Oriented Software* first came out, developers have applied these patterns thousands of times. The patterns we summarize in this appendix are full-fledged, card-carrying, official GoF patterns, but aren't used as often as the patterns we've explored so far. But these patterns are awesome in their own right, and if your situation calls for them, you should apply them with your head held high. Our goal in this appendix is to give you a high-level idea of what these patterns are all about.



Intro



In this section, we answer the burning question:
"So, why DID they put that in a design patterns book?"

Who is this book for?

If you can answer "yes" to all of these:

1. Do you know **Java** (you don't need to be a guru) or another object-oriented language?
2. Do you want to **learn**, **understand**, **remember**, and ***apply*** design patterns, including the OO design principles upon which design patterns are based?
3. Do you prefer **stimulating dinner party conversation** to dry, dull, academic lectures?

All our examples are in Java, but you should be able to understand the main concepts of the book if you know another object-oriented language.

this book is for you.

Who should probably back away from this book?

If you can answer "yes" to any one of these:

1. Are you completely new to **object-oriented programming?**
2. Are you a kick-butt object-oriented designer/ developer looking for a ***reference*** **book**?
3. Are you an architect looking for ***enterprise*** design patterns?
4. Are you **afraid to try something different**? Would you rather have a root canal than mix stripes with plaid? Do you believe that a technical book can't be serious if object-oriented concepts are anthropomorphized?

this book is not for you.

[Note from marketing: this book is for anyone with a credit card.]

We know what you're thinking.

"How can this be a serious programming book?"

"What's with all the graphics?"

"Can I actually learn it this way?"

And we know what your brain is thinking.

Your brain craves novelty. It's always searching, scanning, *waiting* for something unusual. It was built that way, and it helps you stay alive.

Today, you're less likely to be a tiger snack. But your brain's still looking. You just never know.

So what does your brain do with all the routine, ordinary, normal things you encounter? Everything it *can* to stop them from interfering with the brain's *real* job—recording things that matter. It doesn't bother saving the boring things; they never make it past the "this is obviously not important" filter.

How does your brain *know* what's important? Suppose you're out for a day hike and a tiger jumps in front of you, what happens inside your head and body?

Neurons fire. Emotions crank up. *Chemicals surge*.

And that's how your brain knows...

This must be important! Don't forget it!

But imagine you're at home, or in a library. It's a safe, warm, tiger-free zone. You're studying. Getting ready for an exam. Or trying to learn some tough technical topic your boss thinks will take a week, ten days at the most.

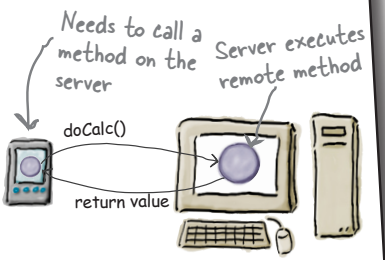
Just one problem. Your brain's trying to do you a big favor. It's trying to make sure that this *obviously* non-important content doesn't clutter up scarce resources. Resources that are better spent storing the really big things. Like tigers. Like the danger of fire. Like how you should never again snowboard in shorts.

And there's no simple way to tell your brain, "Hey brain, thank you very much, but no matter how dull this book is, and how little I'm registering on the emotional Richter scale right now, I really do want you to keep this stuff around."

We think of a "Head First" reader as a learner.

So what does it take to *learn* something? First, you have to *get* it, then make sure you don't *forget* it. It's not about pushing facts into your head. Based on the latest research in cognitive science, neurobiology, and educational psychology, *learning* takes a lot more than text on a page. We know what turns your brain on.

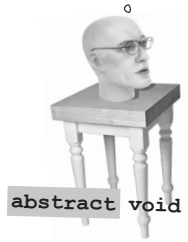
Some of the Head First learning principles:



Make it visual. Images are far more memorable than words alone, and make learning much more effective (up to 89% improvement in recall and transfer studies). It also makes things more understandable. **Put the words within or near the graphics** they relate to, rather than on the bottom or on another page, and learners will be up to *twice* as likely to solve problems related to the content.

Use a conversational and personalized style. In studies, students performed up to 40% better on post-learning tests if the content spoke directly to the reader, using a first-person, conversational style rather than taking a formal tone. Tell stories instead of lecturing. Use casual language. Don't take yourself too seriously. Which would *you* pay more attention to: a stimulating dinner party companion, or a lecture?

It really sucks to be an abstract method. You don't have a body.



```
abstract void roam();
```

No method body! End it with a semicolon.

Get the learner to think more deeply. In other words, unless you actively flex your neurons, nothing much happens in your head. A reader has to be motivated, engaged, curious, and inspired to solve problems, draw conclusions, and generate new knowledge. And for that, you need challenges, exercises, thought-provoking questions, activities that involve both sides of the brain, and multiple senses.

Does it make sense to say Tub IS-A Bathroom? Bathroom IS-A Tub? Or is it a HAS-A relationship?

Get—and keep—the reader's attention. We've all had the "I really want to learn this but I can't stay awake past page one" experience. Your brain pays attention to things that are out of the ordinary, interesting, strange, eye-catching, unexpected. Learning a new, tough, technical topic doesn't have to be boring. Your brain will learn much more quickly if it's not.

Touch their emotions. We now know that your ability to remember something is largely dependent on its emotional content. You remember what you *care* about. You remember when you *feel* something. No, we're not talking heart-wrenching stories about a boy and his dog. We're talking emotions like surprise, curiosity, fun, "what the...?", and the feeling of "I Rule!" that comes when you solve a puzzle, learn something everybody else thinks is hard, or realize you know something that "I'm more technical than thou" Bob from engineering *doesn't*.